AIRFLOW DAG REQUIREMENTS

Daily Insurance Claims Processing

A production-oriented requirements package covering orchestration, Kafka publishing, success and failure handling, and stakeholder impact.

Document scope	Specification
Platform	Apache Airflow
Data domain	Insurance claims
Delivery target	Kafka topic: insurance.claims.processed
People names	Fictional, used for realistic workflow illustration
Prepared	September 12, 2026

Purpose

Define how the daily claims pipeline must ingest, validate, enrich, score, route, publish, monitor, and recover from failures. This document contains requirements only and does not include implementation code.

1. Business and scheduling requirements

Business requirement

Every day, process newly received insurance claims, validate them, enrich them with customer and policy data, calculate risk scores, and publish successfully processed claims to Kafka.

Schedule

- Run daily at 2:00 AM Eastern Time.
- Process data from the previous calendar day.
- Allow manual reruns for a selected date.
- Do not process future dates.
- Allow only one active DAG run at a time.

Input sources

- Claims CSV files from an S3 inbound bucket.
- Customer information from PostgreSQL.
- Policy information from a REST API.
- Expected filename pattern: **claims_YYYYMMDD.csv**.

Workflow requirements

1. Confirm that the expected claims file exists in S3.
2. Validate that the file is not empty.
3. Validate required columns: claim ID, customer ID, policy ID, claim amount, and claim date.
4. Check for duplicate claim IDs.
5. Load valid claim data into a staging area.
6. Retrieve matching customer records from PostgreSQL.
7. Retrieve policy information from the policy API.
8. Join claims with customer and policy information.
9. Calculate a risk score for every claim.
10. Separate successful and rejected claims.
11. Publish successful claims to Kafka.
12. Store rejected claims in the S3 error folder.
13. Update the daily claims summary table.
14. Archive the original input file.
15. Send a completion notification with processing and Kafka totals.

2. End-to-end system workflow

The diagram shows both record-level rejection and platform-level failure. A properly stored rejected claim does not automatically fail the DAG.

3. Kafka publishing requirements

Topic and delivery

- Publish successful claims to **insurance.claims.processed**.
- Use claim ID as the Kafka message key so events for the same claim remain ordered within a partition.
- Require producer acknowledgments from Kafka before the DAG treats a message as published.
- Use at-least-once delivery with idempotent producer behavior.
- Use a managed schema and an explicit schema version.
- Use TLS in transit and approved authentication and authorization controls.

Required Kafka message fields

Field	Purpose
Claim ID	Unique claim identifier and Kafka key
Customer ID	Links the claim to the customer
Policy ID	Links the claim to its policy
Claim date	Business date of the claim
Claim amount	Submitted monetary amount
Risk score	Calculated fraud or operational risk value
Processing status	Successful processing state
Processing timestamp	Time the event was produced
Airflow DAG run ID	Operational traceability
Schema version	Consumer compatibility

Operational controls

- Publish in bounded batches appropriate to broker and message-size limits.
- Retry transient failures without creating logically duplicated outcomes.
- Maintain a stable message key and event identity.
- Store task-relevant credentials in an approved secret manager or Airflow connection.
- Track attempted, acknowledged, failed, and retried messages.

4. Failure handling and data quality

Failure requirements

- Retry Kafka publishing up to three times after the initial attempt.
- Apply a configured retry delay and avoid uncontrolled retry storms.
- Do not archive the original input file when a required downstream publish fails.
- After retry exhaustion, fail the DAG and send an operational alert.
- Write unpublishable events to an approved dead-letter destination when configured.
- Permit a corrected run to be executed for the same logical processing date.

Data quality requirements

- Every successful claim must have one acknowledged Kafka publication outcome.
- Every rejected claim must be stored with a rejection reason.
- Successful count plus rejected count must equal the total input count.
- Acknowledged Kafka count must equal the successful claim count.
- Duplicate claim IDs must be detected before publishing.
- Every event must satisfy the registered message schema.

Success criteria

- All input records are accounted for.
- All valid claims are acknowledged by Kafka.
- All rejected claims are stored with reasons.
- Processing counts reconcile.
- The daily summary is updated.
- The original input file is archived only after required work succeeds.
- The completion notification is sent.

Important distinction

Record rejection is a handled business outcome. The DAG may still succeed when invalid claims are correctly routed to the error location and every valid claim is published. A DAG failure occurs when required platform work cannot complete, such as missing input, unavailable dependencies, unrecoverable validation failure, or Kafka retry exhaustion.

5. Monitoring and notifications

Required metrics

- Total input records.
- Successful and rejected claim counts.
- Kafka attempted and acknowledged counts.
- Kafka failure and retry counts.
- Dead-letter count, when applicable.
- DAG duration and task-level duration.
- Dependency availability and API response failures.

Alert conditions

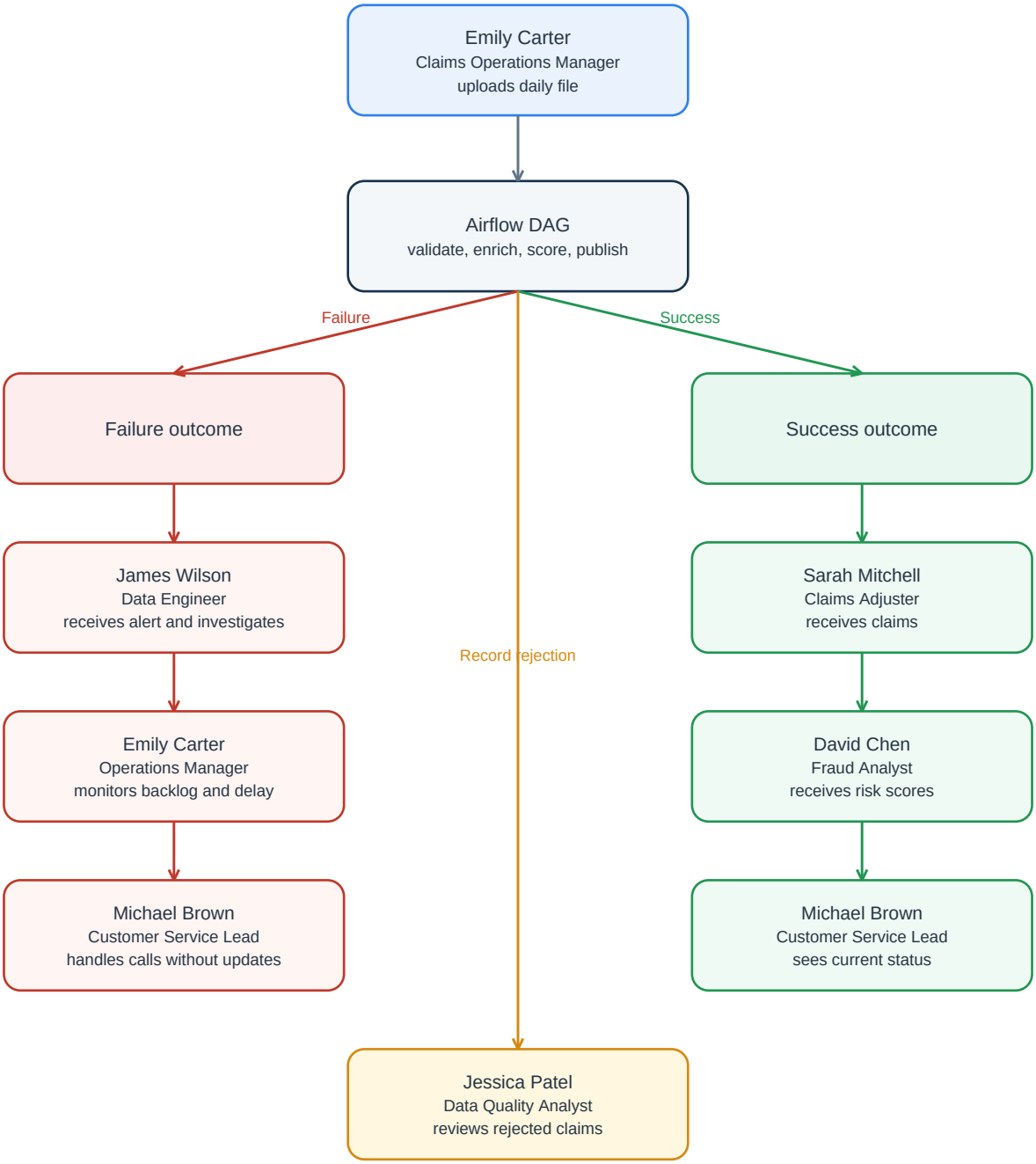
- The expected S3 input file is missing or empty.
- The required file schema is invalid.
- PostgreSQL or policy API enrichment cannot complete.
- Kafka publishing fails after all configured attempts.
- Kafka acknowledgment count differs from the successful claim count.
- The rejected-record rate exceeds the agreed threshold, initially 10 percent.
- The DAG exceeds its operational completion window.

Notifications

Outcome	Audience	Minimum content
Success	Claims Operations	Run date, total, successful, rejected, Kafka acknowledged, duration
Record rejections	Claims Operations and Data Quality	Rejected count, error location, top rejection reasons
DAG failure	Data Engineering and Claims Operations	Failed task, error summary, run ID, retry status, operational impact
Recovery	Data Engineering, Claims Operations, Customer Service	Recovered run ID, restored data date, backlog status

6. People and business impact workflow

The names below are fictional. They make the operational ownership and human impact concrete.



7. Stakeholder impact matrix

Person	Role	Success Impact	Failure Impact
Emily Carter	Claims Operations Manager	Receives reconciled totals and confirms daily processing.	Monitors backlog, reviews alerts, and communicates delays.
Sarah Mitchell	Claims Adjuster	Receives processed claims through downstream consumers.	Cannot begin reviewing delayed claims.
David Chen	Fraud Analyst	Receives enriched claim details and risk scores.	Fraud review is delayed or based on older information.
Michael Brown	Customer Service Lead	Can provide customers with current claim status.	Handles inquiries without updated claim information.
Jessica Patel	Data Quality Analyst	Reviews rejected claims and improves source quality.	Investigates abnormal rejection volume or broken input data.
James Wilson	Data Engineer	Monitors execution and platform performance.	Troubleshoots Airflow, S3, PostgreSQL, API, or Kafka failures.

Operational ownership summary

- Emily owns the business processing outcome and backlog communication.
- James owns technical incident diagnosis and pipeline recovery.
- Jessica owns rejected-record analysis and data quality follow-up.
- Sarah and David depend on timely downstream claim events.
- Michael manages the customer-facing consequences of delayed or unavailable claim status.